SMART PARKING MONITORING SYSTEM

USING COMPUTER VISION

Project Report

Submitted in partial fulfilment of the requirements for
[Course / Degree Name]

Submitted by

[Student Name]
[Roll Number]

Under the guidance of
[Project Guide Name]

[Department Name]
[College / University Name]
Academic Year [20XX-20XX]

Certificate

This is to certify that the project report entitled **"Smart Parking Monitoring System Using Computer Vision"** is a bonafide work carried out by **[Student Name]**, bearing roll number **[Roll Number]**, under my guidance, in partial fulfilment of the requirements for **[Course / Degree Name]** during the academic year **[Year]**.

Project Guide	Head of Department
Name: _____	Name: _____
Signature: _____	Signature: _____
Date: _____	Date: _____

Declaration

I hereby declare that the project entitled **"Smart Parking Monitoring System Using Computer Vision"** is my work, completed under the guidance of **[Guide Name]**. The report has not been submitted to another institution for an academic award. Sources used in this work are acknowledged in the References section.

Student Name: [Name]

Roll Number: [Roll Number]

Signature: _____

Date: _____

Acknowledgement

I would like to thank **[Project Guide Name]** for guidance and feedback throughout this project. I also thank the faculty and staff of **[Department / College Name]** for providing the facilities and support required for this work. Finally, I thank my family and classmates for their encouragement.

Abstract

This project presents an image-based parking-space occupancy monitoring application built with Python, OpenCV, and Streamlit. The application processes a parking-lot image, examines defined parking-space regions, and classifies each region as occupied or vacant. The image-processing pipeline converts the input to grayscale, applies Gaussian blur and adaptive thresholding, and uses median filtering and morphological dilation to create a binary image. For each defined region, the application counts non-zero pixels and compares the count with a configurable threshold. It displays the input image, an annotated result, occupancy counts, and a warning when occupancy reaches a configurable level.

Two supplied reference images have calibrated parking-space layouts. When either reference image is uploaded, the application recognizes it and reuses its saved layout. For another image, the user marks each parking space with a rectangle. The project includes a PKLot dataset export, but the current application occupancy predictions are based on image processing and defined regions; they are not produced by a PKLot-trained classifier. The measured results for the two reference images are checks against labels stored for those same images, not independent test-set accuracy.

Table of Contents

Certificate . . . 2

Declaration . . . 3

Acknowledgement . . . 4

Abstract . . . 5

Table of Contents . . . 6

List of Figures . . . 8

List of Tables . . . 8

Chapter 1: Introduction . . . 9

1.1 Background . . . 9

1.2 Motivation . . . 9

1.3 Scope . . . 9

Chapter 2: Literature Review . . . 10

2.1 Parking Monitoring Approaches . . . 10

2.2 Classical Computer Vision . . . 10

2.3 Dataset-Based Learning . . . 10

2.4 Summary . . . 10

Chapter 3: Problem Statement and Objectives . . . 11

3.1 Problem Statement . . . 11

3.2 Objectives . . . 11

3.3 Assumptions . . . 11

Chapter 4: System Requirements and Technologies Used . . . 12

4.1 Hardware Requirements . . . 12

4.2 Software Requirements . . . 12

4.3 Technologies . . . 12

Chapter 5: Proposed Methodology . . . 13

5.1 Processing Steps . . . 13

5.2 Occupancy Formula . . . 13

Chapter 6: System Architecture and Design . . . 14

6.1 Main Components . . . 14

6.2 Data Flow . . . 14

6.3 Module Separation . . . 14

Chapter 7: Dataset Collection and Preprocessing . . . 15

7.1 Project Reference Images . . . 15

7.2 PKLot Dataset . . . 15

7.3 Preprocessing . . . 15

Chapter 8: Computer Vision Model and Algorithms . . . 16

8.1 Method . . . 16

8.2 Classification Rule . . . 16

8.3 Annotation and Summary . . . 16
8.4 Limitations of the Algorithm . . . 16
Chapter 9: System Implementation . . . 17
9.1 Application Structure . . . 17
9.2 Running the Project . . . 17
9.3 Using the Application . . . 17
Chapter 10: Results and Performance Evaluation . . . 18
10.1 Reference Image Results . . . 18
10.2 Warning Behaviour . . . 18
10.3 Interpretation . . . 18
Chapter 11: Testing and Validation . . . 19
11.1 Validation Notes . . . 19
Chapter 12: Limitations and Future Scope . . . 20
12.1 Limitations . . . 20
12.2 Future Scope . . . 20
Chapter 13: Conclusion . . . 21
References . . . 22
Appendices . . . 23
Appendix A: Suggested Screenshots . . . 23
Appendix B: Parking-Space Configuration . . . 23
Appendix C: Execution Commands . . . 23
Appendix D: FCV Practical Notebook . . . 23

List of Figures

- Figure 1. System architecture (to be inserted by student)
- Figure 2. Image preprocessing stages (to be inserted by student)
- Figure 3. Streamlit upload and result interface (to be inserted by student)
- Figure 4. Annotated occupancy output (to be inserted by student)
- Figure 5. Adaptive-threshold binary visualization (to be inserted by student)

List of Tables

- Table 1. System requirements
- Table 2. Dataset split summary
- Table 3. Reference image results
- Table 4. Functional test cases

Chapter 1: Introduction

1.1 Background

Parking availability information can help drivers locate vacant spaces and help operators monitor a parking area. Manual monitoring becomes difficult when a lot contains many spaces or when occupancy changes frequently. Computer vision provides a way to analyze images from a camera and present occupancy estimates.

This project demonstrates an image-based workflow using classical computer-vision operations. A user provides an image, and the application classifies defined parking-space regions as occupied or vacant. It presents results in a Streamlit interface with per-space coloured rectangles and overall occupancy metrics.

1.2 Motivation

The project provides an interactive and visual way to study how image preprocessing and region-of-interest analysis can be used in a parking application. It avoids requiring physical sensors in each parking bay and exposes the processing steps for educational purposes.

1.3 Scope

The system supports uploaded images, calibrated layouts for two supplied reference images, and manual rectangle selection for other images. It demonstrates image processing and result visualization. Automatic localization of parking bays in arbitrary images is outside the current implementation scope.

Chapter 2: Literature Review

2.1 Parking Monitoring Approaches

Parking monitoring approaches include sensor-based systems and vision-based systems. Sensor-based systems use devices such as ultrasonic or magnetic sensors at individual spaces. Vision-based systems process camera images or video to infer occupancy. Vision systems can cover many spaces from a camera view, but they depend on suitable camera placement and image quality.

2.2 Classical Computer Vision

Classical methods may use filtering, thresholding, edges, texture features, and region-of-interest analysis. These methods are interpretable and useful for teaching. Their results can vary with shadows, lighting, camera position, image quality, and the accuracy of parking-space regions.

2.3 Dataset-Based Learning

Machine-learning approaches learn visual features from labelled examples. The PKLot dataset is relevant to parking-occupancy research because it includes images and labelled parking spaces. The presence of PKLot files in this project does not mean that the application uses a model trained on the dataset. The app prediction path currently uses a threshold-based image-processing method.

2.4 Summary

This project uses an interpretable image-processing baseline and pre-defined or user-drawn space regions. It does not claim general-purpose, automatic detection or performance on all camera views.

Chapter 3: Problem Statement and Objectives

3.1 Problem Statement

Given a parking-area image and the locations of the parking spaces, classify each space as occupied or vacant, annotate the image, and calculate the total spaces, occupied spaces, vacant spaces, and occupancy percentage.

3.2 Objectives

- Provide an interface for uploading parking images.
- Reuse calibrated space layouts for the two supplied reference images.
- Allow users to mark parking spaces in other images.
- Process defined regions using computer-vision operations.
- Classify each region using a configurable pixel-count threshold.
- Display a colour-coded result, occupancy metrics, and an alert threshold.
- Keep the FCV practical notebook separate from the application.

3.3 Assumptions

Each parking space is represented by a rectangle covering the relevant image region. The saved coordinates apply only to the matching reference images. A user must define the regions for another image. The result depends on region selection, image conditions, and the threshold.

Chapter 4: System Requirements and Technologies Used

4.1 Hardware Requirements

- Computer capable of running Python; 4 GB RAM or more recommended.
- Display, keyboard, and mouse or trackpad.
- Parking image in JPG, JPEG, or PNG format.

4.2 Software Requirements

Component	Purpose
Python	Application programming language
OpenCV	Image loading, filtering, thresholding, morphology, and drawing
NumPy	Image arrays and pixel data processing
Streamlit	Interactive web interface
Pillow	Image-format handling and resizing
Streamlit Drawable Canvas	Manual parking-space rectangle selection
Jupyter Notebook	Execution of the separate FCV practicals

4.3 Technologies

The application is implemented in Python. OpenCV provides the image-processing operations; NumPy handles image arrays; Streamlit provides the interface; Pillow supports image handling; and the drawable-canvas component provides interactive region marking.

Chapter 5: Proposed Methodology

The processing flow consists of image input, reference matching or manual region definition, image preprocessing, region analysis, occupancy classification, and visualization.

5.1 Processing Steps

1. The user uploads an image using the file uploader.

2. The app compares the upload with the two bundled reference images. If a match is found, the corresponding saved parking-space layout is loaded.

3. For a non-reference image, the user draws one rectangle around each visible parking space.

4. The image is converted from BGR colour to grayscale.

5. Gaussian blur reduces small variations and noise.

6. Adaptive Gaussian thresholding produces a binary image.

7. Median filtering reduces isolated noise and morphological dilation connects small gaps.

8. The application counts non-zero pixels in each defined region. Large regions use an inset to reduce boundary interference.

9. The count is compared with the configured threshold. A count above the threshold is occupied; otherwise, it is vacant.

10. The app displays coloured annotations, totals, occupancy percentage, and a warning when configured.

5.2 Occupancy Formula

Occupancy percentage is computed as:

Occupancy Percentage = (Occupied Spaces / Total Spaces) x 100

The image page default threshold is 900 non-zero pixels. The default near-full warning threshold is 80 percent.

Chapter 6: System Architecture and Design

6.1 Main Components

- User interface: accepts an uploaded image and displays controls, metrics, and results.
- Image preprocessing: converts the image into a processed binary representation.
- Space-region analysis: loads a saved reference layout or uses user-drawn rectangles.
- Occupancy classification: compares the region pixel count with a threshold.
- Visualization: draws red and green rectangles, displays metrics, and offers an annotated-image download.

6.2 Data Flow

Uploaded image -> reference matching or manual region marking -> preprocessing -> region pixel counting -> occupancy classification -> annotated image and metrics

6.3 Module Separation

The Streamlit interface is implemented in app.py. Core preprocessing and occupancy functions are in parking_detection.py. The eight FCV experiments remain in FCV_Practicals.ipynb.

Chapter 7: Dataset Collection and Preprocessing

7.1 Project Reference Images

The project contains two 1100 x 720 reference parking images, each with a saved layout of 69 parking spaces. The app recognizes the matching image and uses its corresponding layout. The coordinates and stored labels are in parking_data/parking_spaces.json. The labels describe these same reference images; they are not an independent test set.

7.2 PKLot Dataset

The project directory contains a PKLot-derived COCO dataset export. Its annotation files report the following split sizes:

Split	Images	Annotations
Train	8,691	497,856
Validation	2,483	143,316
Test	1,242	70,684

The categories include space-empty and space-occupied. The current app image-analysis path does not load these annotations or use a PKLot-trained model to make occupancy predictions. The dataset is present, but no claim is made here that the deployed app is trained on PKLot.

7.3 Preprocessing

The app applies grayscale conversion, Gaussian blur, adaptive Gaussian thresholding, median filtering, and morphological dilation. Each parking-space region is then examined in the binary result.

Chapter 8: Computer Vision Model and Algorithms

8.1 Method

The current occupancy decision method is a threshold-based image-processing algorithm, not a trained machine-learning classifier. For each region, the application counts white or non-zero pixels in the processed binary image.

8.2 Classification Rule

Let C_i be the non-zero pixel count for space i and tau be the configured threshold:

Status_i = Occupied if C_i > tau; otherwise Status_i = Vacant.

8.3 Annotation and Summary

Occupied spaces are shown with red rectangles, and vacant spaces with green rectangles. The total is the number of defined spaces. Occupancy percentage is calculated from the predicted occupied count.

8.4 Limitations of the Algorithm

Pixel-count thresholds can be affected by lighting, shadows, surface texture, image resolution, camera angle, and rectangle placement. The method is not an automatic parking-bay detector. For arbitrary images, the user must mark the bays.

Chapter 9: System Implementation

9.1 Application Structure

The project separates interface logic from image processing. app.py manages file upload, navigation, controls, result cards, tabs, and downloads. parking_detection.py contains preprocessing, occupancy classification, counting, and drawing logic. FCV_Practicals.ipynb remains a separate notebook.

9.2 Running the Project

From the project directory, install the dependencies and run the Streamlit app:

Command	Action
python -m pip install -r requirements.txt	Install dependencies
python -m streamlit run app.py	Start the dashboard

Open the local URL printed by Streamlit, usually http://localhost:8501.

9.3 Using the Application

1. Open Image Occupancy Analysis.
2. Select Browse files and choose a parking image.
3. For either bundled reference image, its calibrated layout is applied automatically.
4. For another image, draw one rectangle around each parking bay.
5. Review the counts, status warning, and annotated image.
6. Download the annotated result if required.

Chapter 10: Results and Performance Evaluation

10.1 Reference Image Results

Using the current default image-processing threshold and the saved layouts, the app produced the following results:

Reference image	Total	Occupied	Vacant	Occupancy
Reference image 1	69	57	12	82.6%
Reference image 2	69	55	14	79.7%

For each image, the predicted labels matched the 69 labels stored with the corresponding layout. This is a check on the same reference images and their saved regions, not an independent evaluation. It does not demonstrate accuracy on all images or arbitrary parking lots.

10.2 Warning Behaviour

The default occupancy warning threshold is 80 percent. Reference image 1 exceeds the threshold at 82.6 percent and triggers the near-full warning. Reference image 2 is below the threshold at 79.7 percent.

10.3 Interpretation

The numbers above describe these two specific inputs and current settings. Results on other cameras, lots, or image conditions may differ; no universal accuracy claim is made.

Chapter 11: Testing and Validation

Test	Expected behaviour	Observed result
Application startup	Streamlit interface loads	Passed in local smoke test
Reference image 1 upload	Loads 69-space layout and displays 57 occupied / 12 vacant	Passed
Reference image 2 upload	Loads 69-space layout and displays 55 occupied / 14 vacant	Passed
Other image upload	Shows manual rectangle selection interface	Passed in browser test
Unreadable image	Displays an image decoding error	Handled by input validation
Warning threshold	Shows near-full alert above configured threshold	Passed on reference image 1

11.1 Validation Notes

The application workflow was checked on both bundled reference images and the custom-image marking screen. Reference-image agreement with stored labels should not be confused with generalization testing. Additional independent labelled images are needed to measure real-world performance.

Chapter 12: Limitations and Future Scope

12.1 Limitations

- Arbitrary uploaded parking bays are not detected automatically; the user must mark each region.
- The threshold method can be sensitive to lighting, shadows, camera position, occlusion, and pavement texture.
- Saved layouts apply only to the corresponding reference images.
- Labels associated with the reference images are not an independent test set.
- PKLot files exist in the project, but the current app prediction path does not use a model trained on them.
- The results have not been established for all images or all real-world parking lots.

12.2 Future Scope

- Train and integrate a parking-space occupancy classifier using PKLot training data and evaluate it on a separate held-out test split.
- Investigate automatic parking-space localization using a suitable detector trained on labelled boxes.
- Save and reuse layouts for fixed cameras.
- Evaluate on independent labelled images from different cameras, lighting conditions, weather, and parking layouts.
- Improve robustness to perspective, shadows, occlusion, and partially visible spaces.
- Add confidence indicators and a review process for uncertain predictions.

Chapter 13: Conclusion

This project implements an interactive parking-space occupancy demonstration using OpenCV and Streamlit. It preprocesses images, analyzes defined parking-space regions, classifies them through a configurable binary-pixel threshold, and displays coloured annotations and occupancy metrics.

The two bundled reference images use calibrated layouts and produced 57 occupied and 12 vacant spaces for image 1, and 55 occupied and 14 vacant spaces for image 2. Other images require manual marking of parking spaces. PKLot files are present in the project, but the current app predictions are not generated by a PKLot-trained model. The reported reference checks should not be interpreted as evidence that the system works accurately on all images.

References

1. Almeida, P. R. L., Oliveira, L. S., Britto Jr., A. S., Silva Jr., E. J., & Koerich, A. L. (2015). PKLot - A robust dataset for parking lot classification. Expert Systems with Applications, 42(11), 4937-4949.

2. Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools.

3. OpenCV Documentation. Image Processing in OpenCV. https://docs.opencv.org/

4. Streamlit Documentation. Streamlit API Reference. https://docs.streamlit.io/

5. Universidade Federal do Parana. PKLot Dataset. https://web.inf.ufpr.br/vri/databases/pklot/

Appendices

Appendix A: Suggested Screenshots

Insert screenshots of the app interface, the upload workflow, the marked-space canvas for a custom image, and an annotated result. The report PDF intentionally contains no parking reference photographs.

Appendix B: Parking-Space Configuration

The JSON layout stores an image name, image dimensions, and a list of rectangles. Each rectangle contains an identifier and x, y, width, and height coordinates. Reference labels may be included for the bundled examples.

Appendix C: Execution Commands

Purpose	Command
Install dependencies	python -m pip install -r requirements.txt
Run app	python -m streamlit run app.py
Open practical notebook	jupyter notebook FCV_Practicals.ipynb

Appendix D: FCV Practical Notebook

The separate FCV_Practicals.ipynb notebook contains the project practicals. It is not replaced by the Streamlit interface or this report.